

30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 17)

**“Stream Buffers and
Message Buffers”**



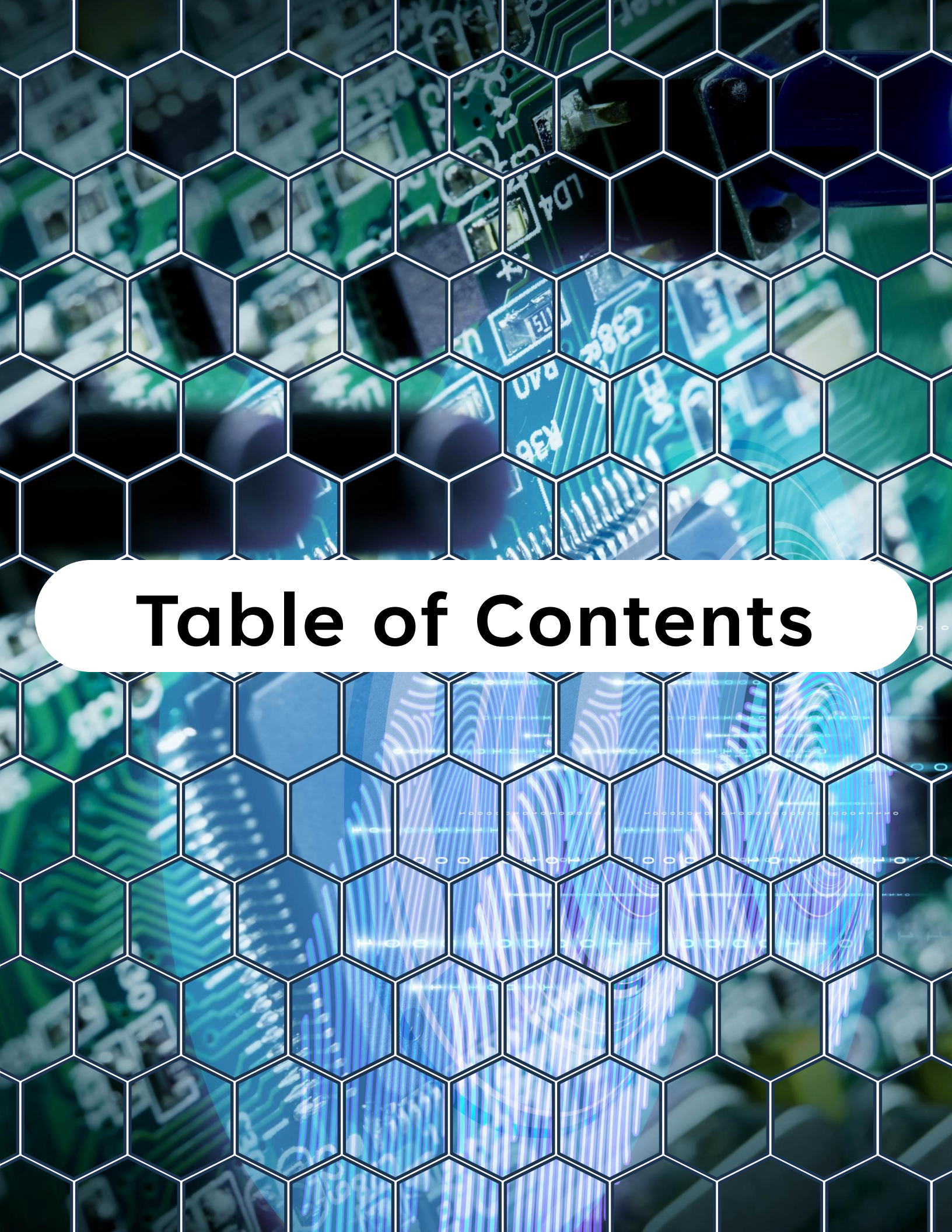
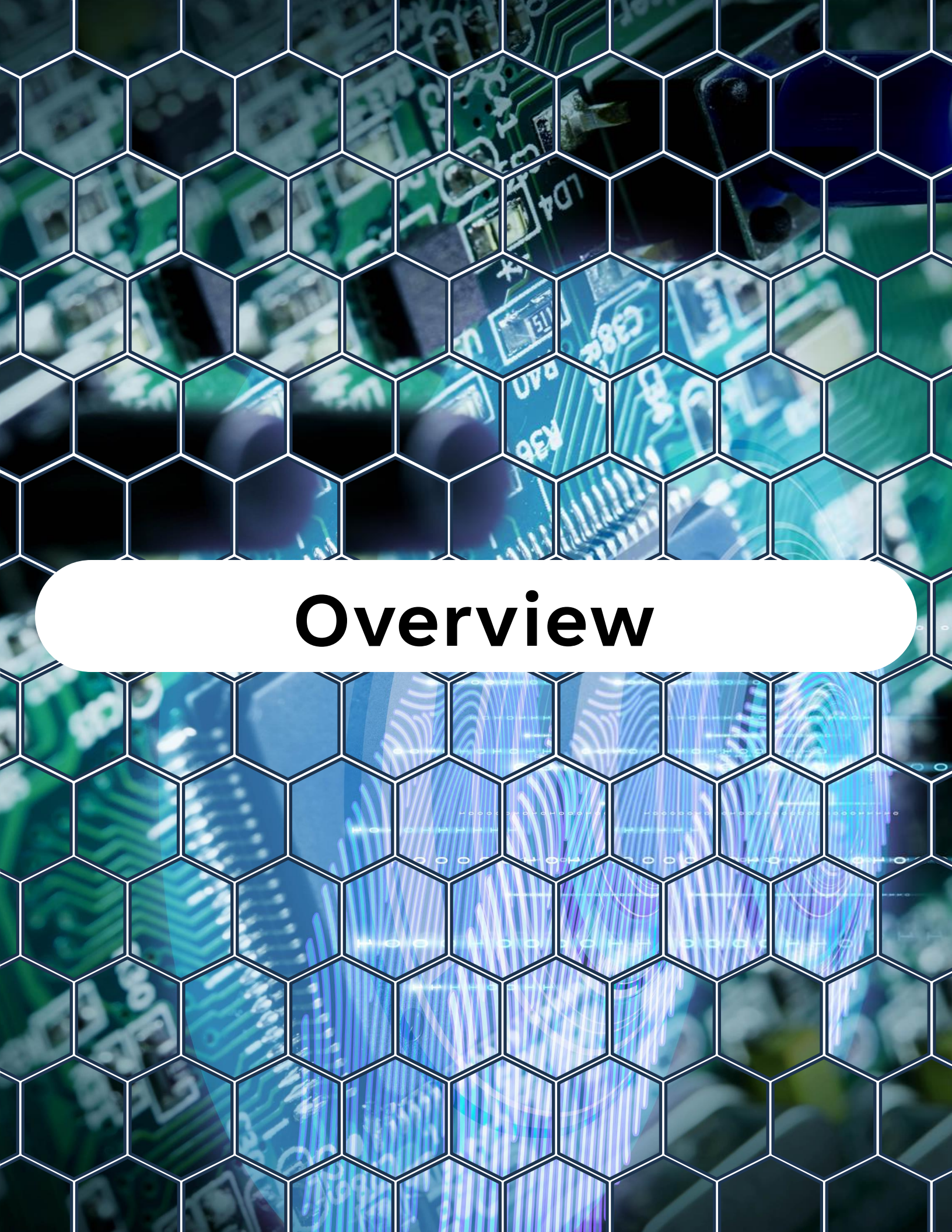


Table of Contents

Table of Contents

1. Overview
2. What Are Stream Buffers and Message Buffers?
3. API Functions Overview
4. Differences from Queues
5. Example – Stream and Message Buffers
6. Best Practices
7. Summary
8. Challenge for Today



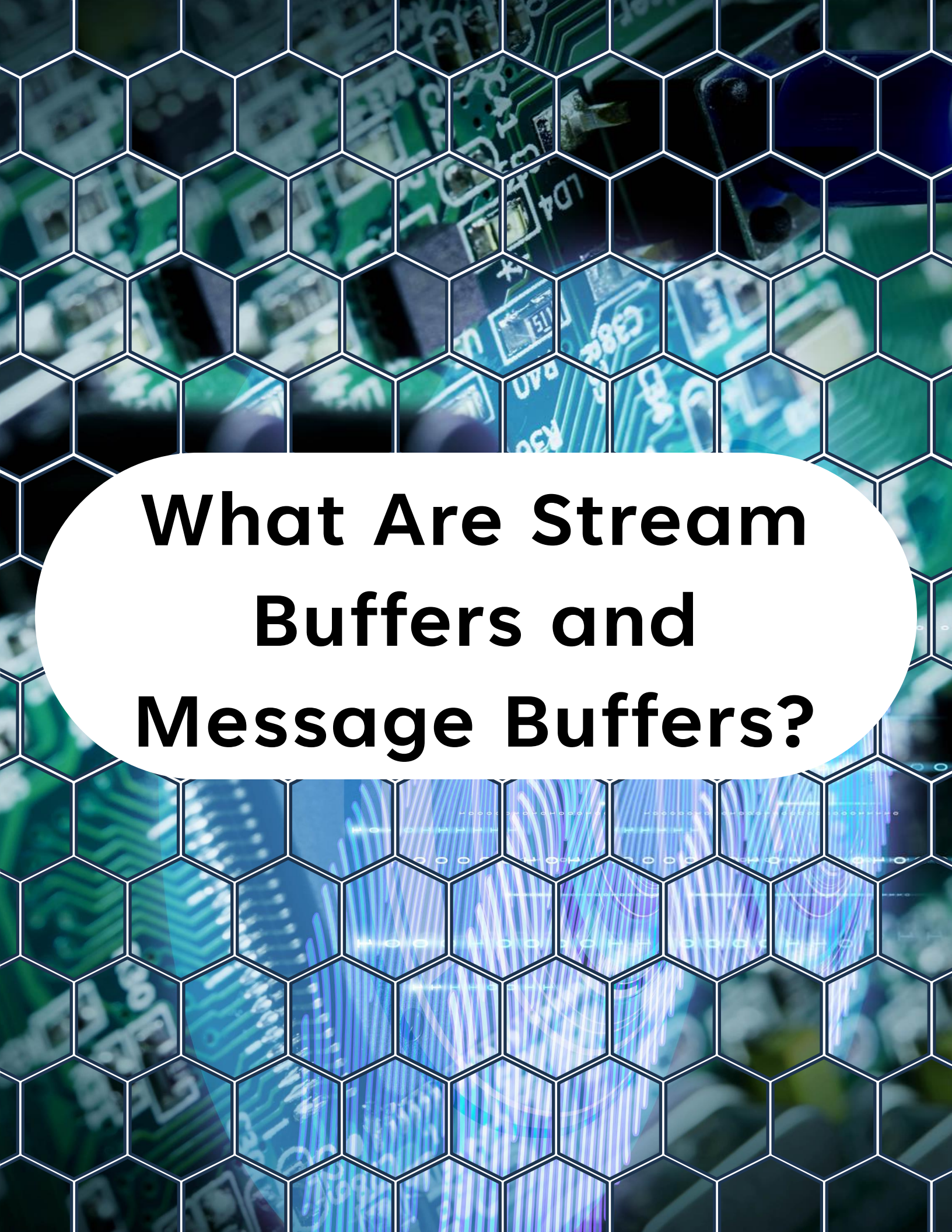
Overview

Overview

On **Day 17**, you'll learn:

- What **stream buffers** and **message buffers** are in FreeRTOS
- How they differ from queues
- How to create, send, and receive data using buffers
- The difference between **stream buffers** (continuous bytes) and **message buffers** (discrete messages)
- A practical ESP32 example using both types
- Best practices for buffer usage

By the end of this lesson, you'll understand how to use these buffers for **efficient data transfer** in FreeRTOS.



What Are Stream Buffers and Message Buffers?

What Are Stream Buffers and Message Buffers?

Both stream buffers and message buffers are designed for task-to-task or ISR-to-task communication. They are lighter and faster than queues when dealing with continuous data.

Stream Buffers

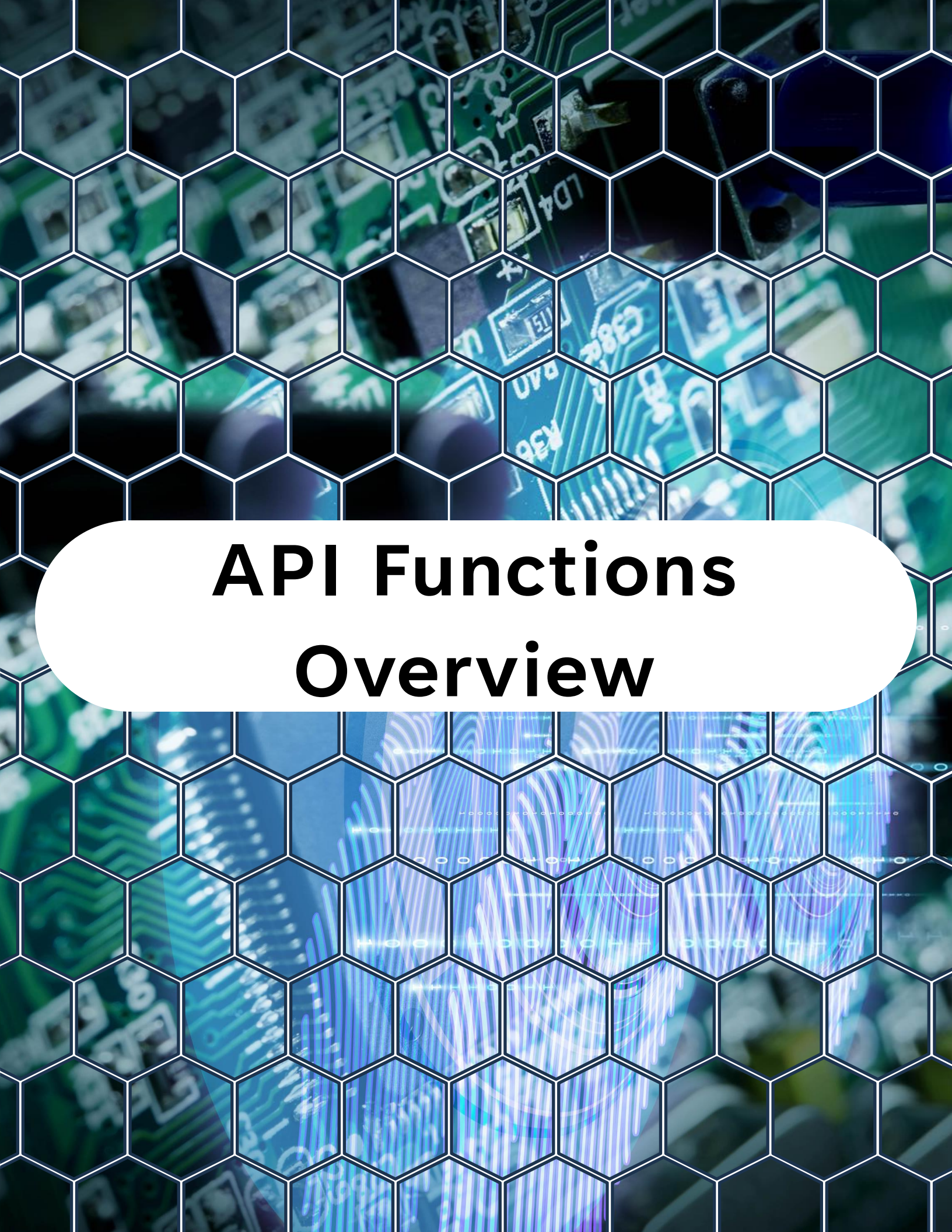
- Store a **stream of bytes** (like a pipe).
- Suitable for **continuous data** (e.g., UART, SPI, ADC samples).
- The receiver just reads bytes as they come.

What Are Stream Buffers and Message Buffers?

Message Buffers

- Store **discrete messages** (each with a length).
- Suitable for **packets** or **framed data** (e.g., network packets, structured messages).
- Receiver always reads a whole message.

 **Both are one-to-one communication mechanisms** (one writer, one reader).



API Functions Overview

API Functions Overview

Create



```
1 StreamBufferHandle_t xStreamBufferCreate(  
2     size_t xBufferSizeBytes, size_t xTriggerLevelBytes);  
3  
4 MessageBufferHandle_t xMessageBufferCreate(  
5     size_t xBufferSizeBytes);
```

- `xBufferSizeBytes`: size of buffer in bytes.
- `xTriggerLevelBytes`: minimum number of bytes before reader task unblocks.

API Functions Overview

Send

```
1  size_t xStreamBufferSend(  
2      StreamBufferHandle_t xStreamBuffer,  
3      const void *pvTxData,  
4      size_t xDataLengthBytes,  
5      TickType_t xTicksToWait);  
6  
7  size_t xMessageBufferSend(  
8      MessageBufferHandle_t xMessageBuffer,  
9      const void *pvTxData,  
10     size_t xDataLengthBytes,  
11     TickType_t xTicksToWait);
```

- **xStreamBuffer**: The handle of the stream buffer to which data is being sent.
- **xMessageBuffer**: The handle of the message buffer to which data is being sent.
- **pvTxData**: Pointer to the data that is being sent to the stream buffer.
- **xDataLengthBytes**: The length of the data being sent, in bytes.
- **xTicksToWait**: The maximum time to wait for space to become available

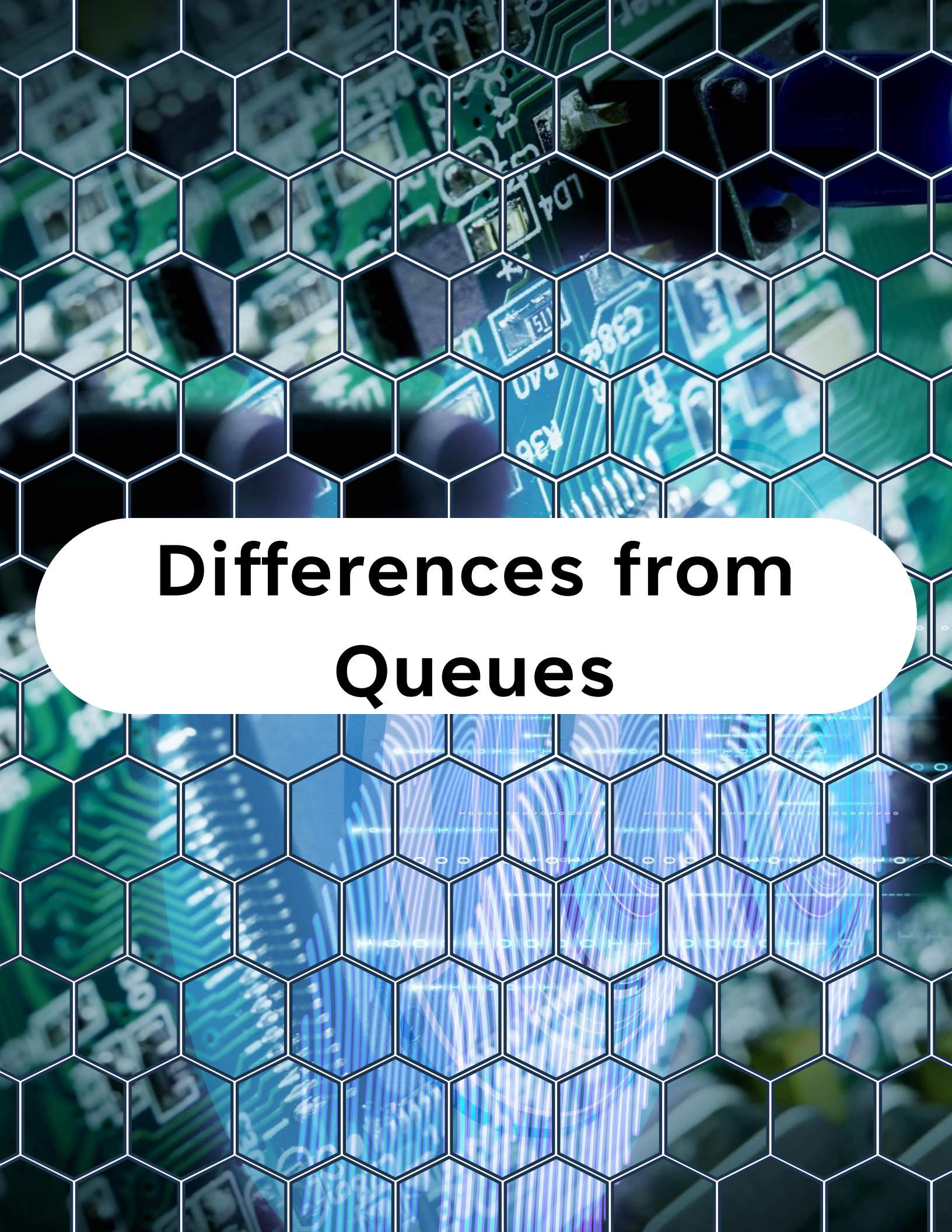
API Functions Overview

Receive



```
1  size_t xStreamBufferReceive(  
2      StreamBufferHandle_t xStreamBuffer,  
3      void *pvRxData,  
4      size_t xBufferLengthBytes,  
5      TickType_t xTicksToWait);  
6  
7  size_t xMessageBufferReceive(  
8      MessageBufferHandle_t xMessageBuffer,  
9      void *pvRxData,  
10     size_t xBufferLengthBytes,  
11     TickType_t xTicksToWait);
```

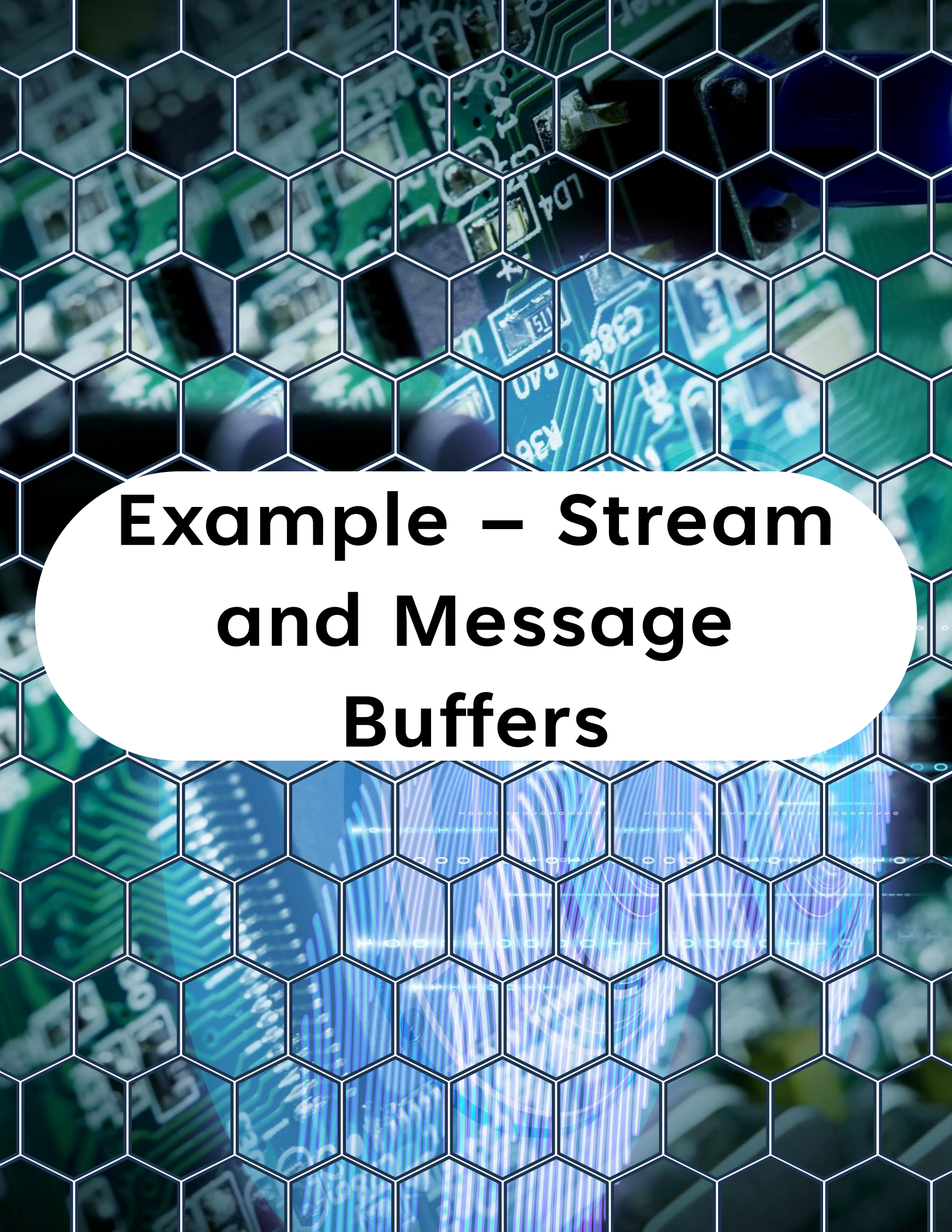
- **xStreamBuffer**: Handle to the stream buffer from which data is to be received.
- **xMessageBuffer**: Handle to the message buffer from which data is to be received.
- **pvRxData**: Pointer to the buffer into which received data will be copied.
- **xBufferLengthBytes**: Length of the buffer pointed to by pvRxData, in bytes.
- **xTicksToWait**: Maximum time to wait for data to be available in the stream buffer, in ticks



Differences from Queues

Differences from Queues

Feature	Queues	Stream Buffers	Message Buffers
Data Type	Fixed-size items	Raw byte stream	Variable-length messages
Overhead	Higher (per item)	Lower (just bytes)	Moderate (length + data)
One-to-One / Many	One-to-one or many	One writer, one reader	One writer, one reader
Use Case	Sensor structs, commands	UART, ADC, audio streaming	Packets, structured messages



Example – Stream and Message Buffers

Example – Stream and Message Buffers

```
1  #include <stdio.h>
2  #include <string.h>
3  #include "freertos/FreeRTOS.h"
4  #include "freertos/task.h"
5  #include "freertos/stream_buffer.h"
6  #include "freertos/message_buffer.h"
7
8  #define TAG "DAY17"
9
10 // Stream buffer handle for sending and receiving raw byte data between tasks
11 StreamBufferHandle_t stream_buf;
12
13 // Message buffer handle for sending and receiving messages between tasks
14 MessageBufferHandle_t msg_buf;
15
16 // Producer task to send data to both stream buffer and message buffer
17 void producer_task(void *pvParameter) {
18     const char *stream_data = "ABCDEF";
19     const char *msg1 = "Hello";
20     const char *msg2 = "World";
21
22     while (1) {
23         // Send raw bytes to stream buffer
24         xStreamBufferSend(stream_buf, stream_data, 6, portMAX_DELAY);
25         printf("Producer: Sent bytes to stream buffer\n");
26
27         // Send messages to message buffer
28         xMessageBufferSend(msg_buf, msg1, strlen(msg1) + 1, portMAX_DELAY);
29         xMessageBufferSend(msg_buf, msg2, strlen(msg2) + 1, portMAX_DELAY);
30         printf("Producer: Sent two messages to message buffer\n");
31
32         vTaskDelay(pdMS_TO_TICKS(2000));
33     }
34 }
35
36 // Consumer task to receive data from both stream buffer and message buffer
37 void consumer_task(void *pvParameter) {
38     char rx_stream[20];
39     char rx_msg[20];
40
```

Example – Stream and Message Buffers

```
36 // Consumer task to receive data from both stream buffer and message buffer
37 void consumer_task(void *pvParameter) {
38     char rx_stream[20];
39     char rx_msg[20];
40
41     while (1) {
42         // Receive from stream buffer
43         size_t received = xStreamBufferReceive(
44             stream_buf, rx_stream, sizeof(rx_stream), pdMS_TO_TICKS(1000));
45         if (received > 0) {
46             rx_stream[received] = '\0';
47             printf("Consumer: Stream buffer received: %s\n", rx_stream);
48         }
49
50         // Receive from message buffer
51         received = xMessageBufferReceive(
52             msg_buf, rx_msg, sizeof(rx_msg), pdMS_TO_TICKS(1000));
53         if (received > 0) {
54             printf("Consumer: Message buffer received: %s\n", rx_msg);
55         }
56     }
57 }
58
59 // Entry point for the application
60 void app_main() {
61     // Create a stream buffer (50 bytes, unblock when at least 1 byte available)
62     stream_buf = xStreamBufferCreate(50, 1);
63
64     // Create a message buffer (100 bytes)
65     msg_buf = xMessageBufferCreate(100);
66
67     // Create tasks
68     xTaskCreate(producer_task, "Producer", 2048, NULL, 5, NULL);
69     xTaskCreate(consumer_task, "Consumer", 2048, NULL, 5, NULL);
70 }
```

Example – Stream and Message Buffers

Expected Behavior

Serial output:

Producer: Sent bytes to stream buffer

Producer: Sent two messages to message buffer

Consumer: Stream buffer received: ABCDEF

Consumer: Message buffer received: Hello

Consumer: Message buffer received: World

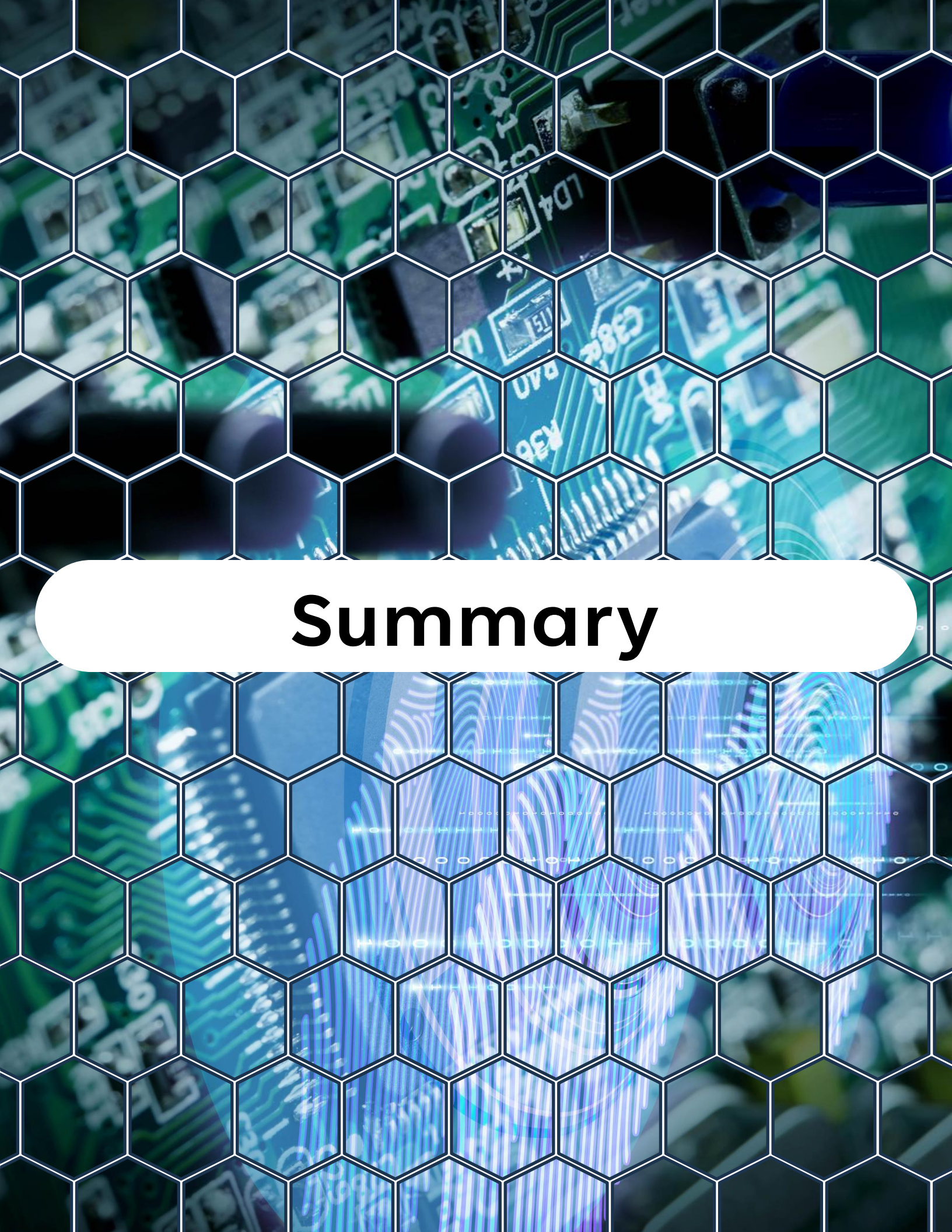
- Stream buffer delivers raw bytes (ABCDEF).
- Message buffer delivers discrete messages (Hello, World).



Best Practices

Best Practices

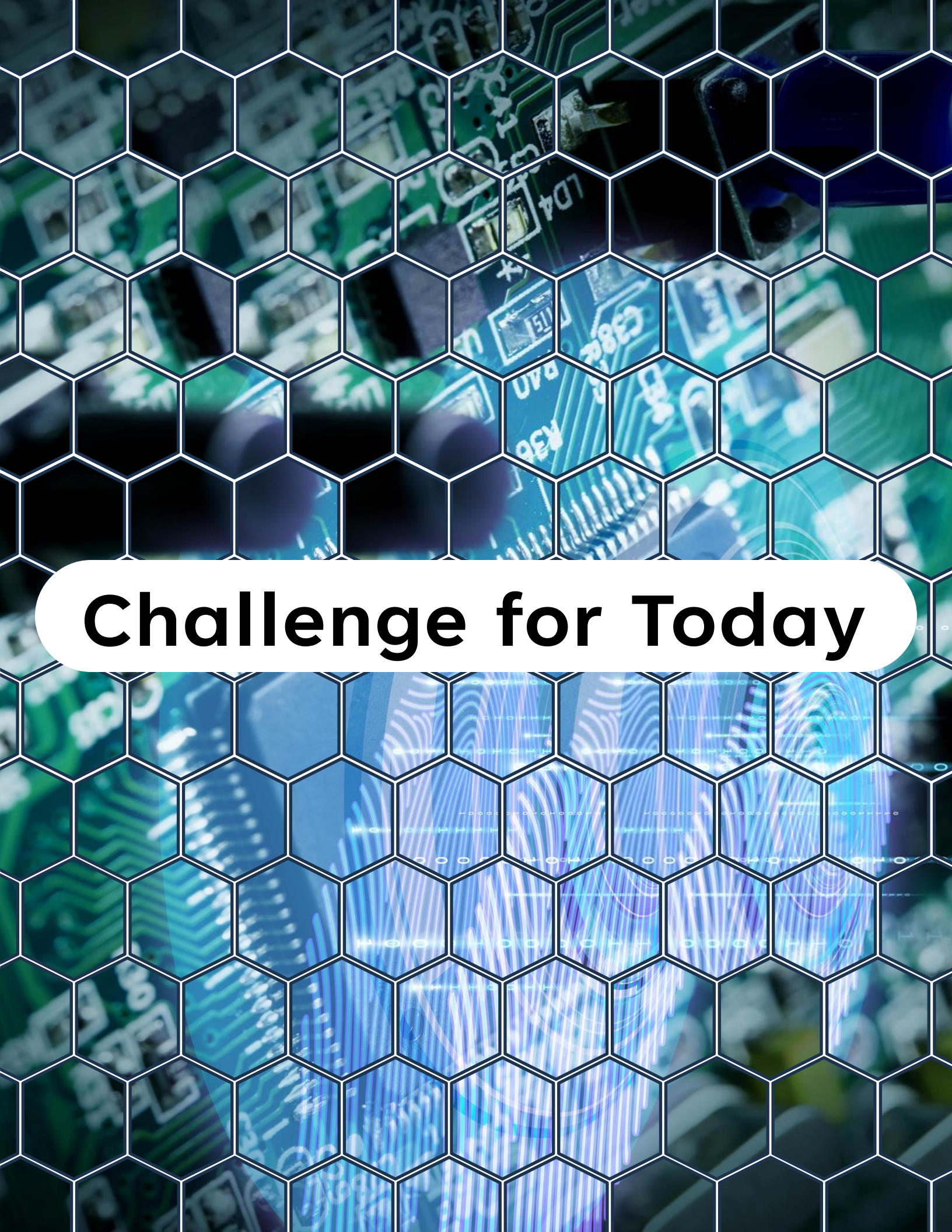
- ✓ Use **stream buffers** for continuous byte streams (UART, ADC, audio).
- ✓ Use **message buffers** for variable-length structured messages.
- ✓ Only **one writer and one reader** per buffer (if you need many-to-one, use queues).
- ✓ Keep buffer sizes aligned with expected traffic to avoid blocking.
- ✓ For ISRs, use **FromISR** versions of send/receive.



Summary

Summary

- **Stream buffers:** continuous byte transfer, one-to-one.
- **Message buffers:** discrete packets/messages, one-to-one.
- Both are lighter than queues for raw data transfer.
- Perfect for efficient communication between producer/consumer tasks.



Challenge for Today

Challenge for Today



Extend today's project by:

1. Adding an **ISR simulation task** that uses `xStreamBufferSendFromISR()` to push bytes.
2. Modify the consumer to **count how many messages** it processed.
3. Try reducing buffer sizes to see how blocking behavior works.

"Thanks for watching!

If you enjoyed this video,

*make sure to hit the like button and
subscribe to stay updated with my latest
content.*

*Don't forget to check out my other
videos for more tips and tutorials on
Embedded C, Python, hardware designs,
etc. Keep exploring, keep learning, and
I'll see you in the next video!"*



<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>